Red Social Estudiantil Pascualina

Tarea 4: Implementación Multi-SGBD, Extensión NoSQL, Big Data e IoT

AUTOR:
Jonathan Andrés Serna Velasco

CURSO:
Bases de Datos I

INSTITUCIÓN:
Institución Universitaria Pascual Bravo

1 INTRODUCCIÓN Y OBJETIVOS DEL PROYECTO

El presente documento constituye la memoria técnica final del desarrollo de la base de datos para la **Red Social Estudiantil Pascualina**. Tras haber modelado el esquema conceptual y lógico en las etapas anteriores, esta Tarea 4 amplía y adapta la arquitectura para soportar tres Sistemas Gestores de Bases de Datos (SGBD) distintos: **PostgreSQL 16**, **MySQL 8.0** y **MongoDB 7.0**.

Adicionalmente, se integra el paradigma NoSQL mediante tipos de datos semiestructurados (`JSON` / `JSONB`) y modelos orientados a documentos BSON, respondiendo a requerimientos reales de captura masiva de datos (*Big Data*) e interacción con dispositivos inteligentes y geolocalización (*IoT*).

2 SELECCIÓN Y JUSTIFICACIÓN DE LOS TRES SGBD

Para garantizar un análisis comparativo riguroso de paradigmas de almacenamiento, se seleccionaron tres motores ampliamente utilizados en la industria tecnológica:

SGBD	Paradigma y Tipo	Ventajas Clave	Rol en la Arquitectura
PostgreSQL 16	Relacional Objeto (RDBMS) + Híbrido NoSQL	Soporte nativo de `JSONB` indexable vía GIN, rigor en llaves foráneas y cumplimiento del estándar SQL.	Motor principal de la base de datos relacional y catálogo institucional.
MySQL 8.0	Relacional (RDBMS)	Motor InnoDB con soporte para tipo `JSON` nativo, columnas generadas persistentes e índices funcionales.	SGBD relacional alternativo optimizado para lecturas rápidas y alta concurrencia.
MongoDB 7.0	NoSQL Orientado a Documentos	Esquema dinámico BSON, escalabilidad horizontal, índices geoespaciales 2dsphere para sensores IoT.	Almacén especializado para flujo de eventos, interacciones en tiempo real y Big Data.

3. CASOS DE USO: INTEGRACIÓN DE NOSQL, BIG DATA E IOT

Las plataformas sociales universitarias modernas no solo gestionan datos estructurados tabulares (como matrículas o listas de estudiantes), sino también flujos continuos e impredecibles de información proveniente de dispositivos móviles y sensores del campus. Para resolver esta necesidad se definieron dos casos de uso de Big Data e IoT:

> **Caso de Uso 1 : Telemetría de Eventos e IoT en el Campus**
>
> Durante la realización de talleres, hackatones o reuniones de estudio presenciales, los dispositivos IoT (sensores de aforo, beacons de proximidad y GPS móvil) emiten coordenadas geográficas, intensidad de señal y conteo de asistentes en tiempo real. Almacenar estas estructuras cambiantes en columnas relacionales tradicionales requeriría continuas modificaciones de esquema (`ALTER TABLE`). La columna `JSONB` / documento BSON permite guardar la telemetría dinámicamente.

> **Caso de Uso 2: Metadatos Flexibles y Analítica de Publicaciones (Big Data)**
>
> Cada tipo de publicación (recursos, memes, preguntas o noticias) contiene metadatos totalmente distintos: resolución y *bitrate* en videos, atributos EXIF en imágenes, o métricas de interacción analítica (vistas únicas, tasa de rebote, tiempo promedio de lectura). El formato JSON en la tabla `publicacion` permite unificar estos datos sin crear múltiples tablas hijas vacías.

4. IMPLEMENTACIÓN DEL CAMPO NOSQL EN POSTGRESQL

A continuación se muestra la instrucción DDL aplicada sobre la tabla `publicacion` en PostgreSQL para incorporar el tipo de dato `JSONB` y su índice especializado GIN:

```
-- Modificación de la tabla publicacion para soporte NoSQL / IoT
ALTER TABLE red_social.publicacion
ADD COLUMN datos_nosql JSONB;

-- Creación de índice GIN para optimizar búsquedas dentro del documento JSONB
CREATE INDEX idx_publicacion_datos_nosql
ON red_social.publicacion USING gin (datos_nosql);

-- Inserción de ejemplo con estructura de telemetría IoT y métricas de Big Data
INSERT INTO red_social.publicacion (
    id_usuario_autor, tipo_publicacion, contenido, datos_nosql
) VALUES (
    1, 'recurso', 'Ubicación y telemetría del taller de robótica.',
    '{
        "iot_telemetria": {
            "sensor_id": "PASCUAL-BEACON-04",
            "ubicacion_geo": {"lat": 6.2731, "lng": -75.5828},
            "dispositivos_cercanos": 38
        },
        "big_data_metadata": {
            "dispositivo_origen": "Android 14",
            "vistas_unicas": 1420,
            "tiempo_lectura_seg": 32.5
        }
    }'::jsonb
);
```

5. IMPLEMENTACIÓN EN MYSQL 8.0 Y MONGODB 7.0

5.1 Script DDL en MySQL (Relacional con Columna Generada)

En MySQL 8.0 se utiliza el tipo de dato `JSON` nativo. Para acelerar las consultas sobre atributos embebidos (como la latitud de un sensor IoT), se añade una columna generada e indexada:

```
-- Script DDL para MySQL 8.0
CREATE DATABASE IF NOT EXISTS red_social_pascualina;
USE red_social_pascualina;

CREATE TABLE publicacion (
    id_publicacion BIGINT AUTO_INCREMENT PRIMARY KEY,
    id_usuario_autor INT NOT NULL,
    tipo_publicacion VARCHAR(20) NOT NULL,
    contenido TEXT NOT NULL,
    fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    datos_nosql JSON,
    CONSTRAINT ck_tipo CHECK (tipo_publicacion IN ('pregunta', 'recurso', 'noticia',
'meme'))
) ENGINE=InnoDB;

-- Columna generada para extraer coordenadas IoT del documento JSON
ALTER TABLE publicacion
ADD COLUMN latitud DOUBLE GENERATED ALWAYS AS
(CAST(datos_nosql->'$.iot_telemetria.ubicacion_geo.lat' AS DOUBLE)) STORED;

CREATE INDEX idx_latitud ON publicacion(latitud);
```

5.2 Script Schema / NoSQL en MongoDB 7.0 (Colección y Validación)

En MongoDB se modela la entidad como una colección de documentos BSON con validación de esquema estricta y un índice espacial de tipo `2dsphere`:

```
// Script de inicialización para MongoDB 7.0
db = db.getSiblingDB('red_social_pascualina');

db.createCollection("publicaciones", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["id_usuario_autor", "tipo_publicacion", "contenido"],
         properties: {
            id_usuario_autor: { bsonType: "int" },
            tipo_publicacion: { enum: ["pregunta", "recurso", "noticia", "meme"] },
            contenido: { bsonType: "string" },
            datos_nosql: { bsonType: "object" }
         }
      }
   }
});

// Índice geoespacial para consultas de proximidad IoT
db.publicaciones.createIndex({ "datos_nosql.iot_telemetria.ubicacion_geo": "2dsphere"
});
```

6. DICCIONARIO DE DATOS FÍSICO COMPARATIVO MULTI-SGBD

A continuación se presenta la homologación de los tipos de datos lógicos traducidos a la implementación física en los tres SGBD evaluados:

Campo Lógico	PostgreSQL 16	MySQL 8.0	MongoDB 7.0	Justificación Técnica
`id_usuario`	INTEGER IDENTITY	INT AUTO_INCREMENT	ObjectId (`_id`)	Clave autoincremental en SQL / Hash disperso de 12 bytes en NoSQL.
`id_publicacion`	BIGINT IDENTITY	BIGINT AUTO_INCREMENT	ObjectId (`_id`)	BIGINT previene el agotamiento de IDs por alto volumen de publicaciones.
`contenido`	TEXT	TEXT	string	Texto de longitud variable sin restricción rígida de caracteres.
`fecha_publicacion`	TIMESTAMP	TIMESTAMP	Date	Marca temporal con precisión de fecha y hora del servidor.
`datos_nosql`	JSONB	JSON	Object (BSON)	Almacenamiento binario estructurado e indexable para IoT/Big Data.
`leido`	BOOLEAN	TINYINT(1)	bool	Representación de valor lógico binario (verdadero/falso).

7. ANÁLISIS DE DIFICULTADES, VENTAJAS Y DESVENTAJAS

PostgreSQL (JSONB): Presenta un rendimiento sobresaliente en consultas indexadas GIN, aunque requiere una sintaxis de extracción de operadores (`->>`) más compleja.

MySQL (JSON): Facilita la extracción de atributos con la sintaxis de flecha (`->`) y columnas generadas, pero el consumo de almacenamiento en disco es mayor al no compilar a formato binario optimizado como JSONB.

MongoDB (BSON): Ofrece la máxima flexibilidad y velocidad de inserción para grandes volúmenes de datos (Big Data / IoT), pero carece del control estricto de integridad referencial nativo que garantizan los motores SQL mediante llaves foráneas.

8. CONCLUSIONES E IMPACTO PROFESIONAL

Aprendizaje del Paradigma Híbrido: La realización de este proyecto permitió comprender que los sistemas modernos no deben ser exclusivamente relacionales ni exclusivamente NoSQL. La combinación de la solidez ACID de un RDBMS con campos dinámicos `JSONB` ofrece la flexibilidad necesaria para resolver retos reales de IoT y Big Data sin renunciar a la integridad referencial.

Adaptación entre Motores: Traducir una misma arquitectura a PostgreSQL, MySQL y MongoDB evidencia las fortalezas técnicas de cada SGBD y proporciona el criterio necesario para seleccionar el motor adecuado según el tipo de carga de trabajo.